Missing Data and Imputation in Python

A concise starter guide for Civil Engineering students using Jupyter Notebook or Google Colab

What this guide covers

- What missing data means and why it matters before analysis or machine learning
- MCAR, MAR, and MNAR with civil engineering examples
- Quick checks in pandas
- Beginner-friendly imputation methods in pandas and scikit-learn
- A safe workflow that avoids data leakage

Software note

All code blocks are intentionally simple and run in ordinary notebook cells. The examples target Python 3.12+ and use pandas, NumPy, matplotlib, and scikit-learn. You can paste them directly into Jupyter Notebook or Google Colab.

Prepared for practical coursework and early project work in Civil Engineering

1. Why missing data matters

In civil engineering data, missing values appear everywhere: missing slump measurements, broken sensor readings, absent weather records, skipped field tests, or survey responses that were never entered.

You should not rush to fill missing entries immediately. First ask why the values are missing, how many are missing, and whether the variable is numeric, categorical, time-ordered, or spatial.

A poor imputation choice can flatten real variability, create bias, or leak information from the test set into the training process.

How missing values appear in Python

Marker	Common place	Example	Note
np.nan	numeric columns	float('nan')	Common missing marker in numeric data
None	object columns	['A', None]	pandas usually treats it as missing
pd.NA	nullable pandas dtypes	dtype="Int64"	Useful for nullable integer, string, and boolean columns
NaT	date/time columns	missing timestamp	Used for missing datetime values

Table 1. Common missing-value markers in beginner Python work. Source: adapted from pandas missing-data documentation [2].

First checks in pandas

Start with simple counts and percentages. These two lines often tell you more than an immediate imputation.

```
import pandas as pd

df.isna().sum()                # number of missing values in each column
100 * df.isna().mean()         # percent missing in each column
```

Listing 1. A minimal first look at missingness in a notebook. pandas recommends isna() and notna() for detecting missing values [2].

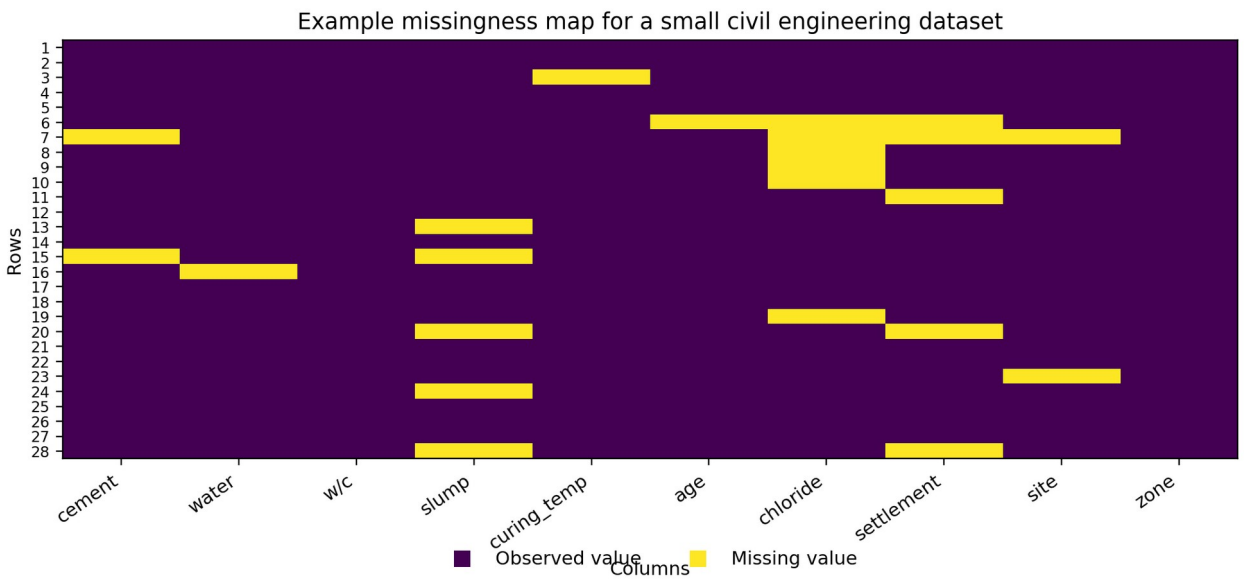

Figure 1. Example missingness map for a small civil engineering dataset. Bright cells represent missing values. Source: author-created synthetic example.

2. The three types of missing data

Before choosing an imputation method, think about the likely mechanism. The classic categories are missing completely at random (MCAR), missing at random (MAR), and missing not at random (MNAR).

Type	Meaning	Civil example	Practical note
MCAR	Missingness is random.	A few concrete cylinders were lost in transport.	Dropping rows is less risky, but you still lose information.
MAR	Missingness depends on observed information.	One site recorded fewer field tests than another site.	Use observed variables that explain the missingness when possible.
MNAR	Missingness depends on the missing value itself.	Very large settlement values are less likely to be reported.	Simple imputation can still leave bias. Domain judgment matters.

Table 2. The three common missing-data mechanisms. Definitions are based on the NCBI overview of missing-data types [1].

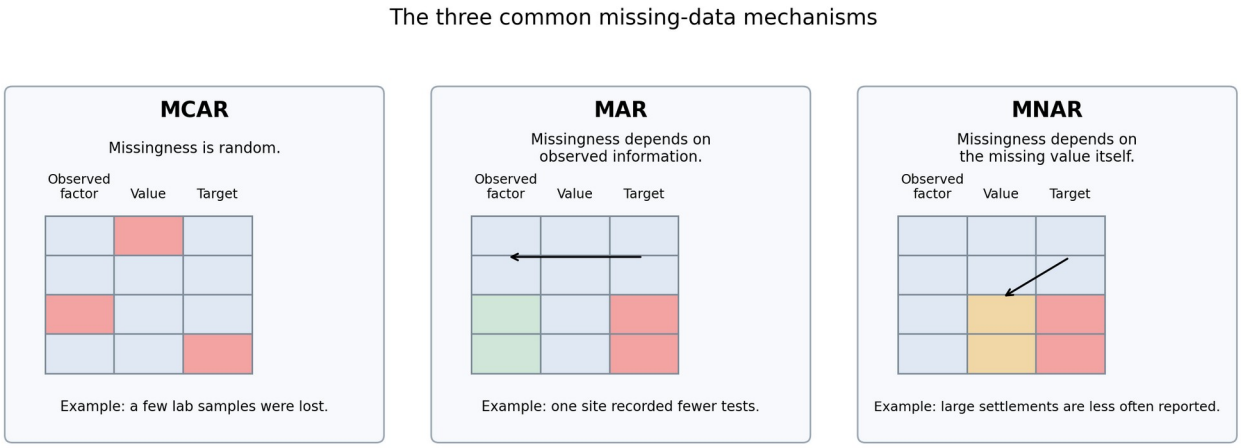

Figure 2. Concept sketch of MCAR, MAR, and MNAR. Source: author-created summary based on [1].

3. A good beginner workflow

Use this order before you fill anything:

1) inspect the amount of missingness, 2) inspect the pattern, 3) decide whether row deletion is acceptable, 4) choose a simple imputer that respects the variable type and data structure, and 5) document what you did.

Quick rule of thumb

- Numeric column with a few values missing and some outliers: try median.
- Categorical column: try most frequent value, or create an explicit 'Unknown' category.
- Time series or ordered measurements: interpolation may be more natural than a single mean.
- When nearby rows are similar: KNN imputation can help, but only after simple checks.
- If missingness may carry information, add a separate indicator column.

When simple deletion is acceptable

Dropping rows or columns is not always wrong. It can be acceptable when the missing fraction is very small, the missingness looks roughly random, and the remaining sample is still large enough for the task.

```
# remove rows with missing target values
df = df.dropna(subset=["strength"])

# remove a column with too many missing entries
df = df.drop(columns=["old_sensor_3"])
```

Listing 2. Very basic deletion with pandas. Use carefully because information is discarded.

Beginner-friendly imputation methods

Method	Good for	Example	Watch out
Constant fill	codes and labels	soil_type -> 'Unknown'	Can create an artificial category
Mean	roughly symmetric numeric data	small lab variables	Sensitive to outliers
Median	skewed numeric data	strength, settlement	Ignores relationships with other variables
Mode	categorical columns	exposure class	Can over-repeat the biggest class
Forward / backward fill	ordered readings	hourly sensor records	Only suitable when order matters
Linear interpolation	time series	daily settlement	Bad for sudden jumps or event-driven changes
KNN imputation	similar nearby rows	mix design rows	Needs sensible scaling and enough similar data
Iterative imputation	strong relations among features	correlated lab variables	More advanced and slower

Table 3. A practical comparison of common beginner-friendly choices.

4. Simple imputation in pandas

pandas is a good place to start because the code is short and easy to read.

Fill numeric columns with mean or median

```
# fill one numeric column with its median
df["settlement_mm"] = df["settlement_mm"].fillna(df["settlement_mm"].median())

# fill one numeric column with its mean
df["temperature_C"] = df["temperature_C"].fillna(df["temperature_C"].mean())
```

Listing 3. Median is often safer than mean when the column has outliers.

Fill categorical columns

```
# use the most common category
df["soil_class"] = df["soil_class"].fillna(df["soil_class"].mode()[0])

# or use an explicit label
df["soil_class"] = df["soil_class"].fillna("Unknown")
```

Listing 4. Simple categorical imputation in pandas.

Interpolation for ordered data

For time-ordered measurements, interpolation is often more defensible than a single mean. This is common for settlement, pore pressure, temperature, or deflection records.

```
# make sure the rows are in time order first
df = df.sort_values("day")

# then interpolate the missing values
df["settlement_mm"] = df["settlement_mm"].interpolate(method="linear")
```

Listing 5. A very simple interpolation example. pandas provides interpolate() for this purpose [3].

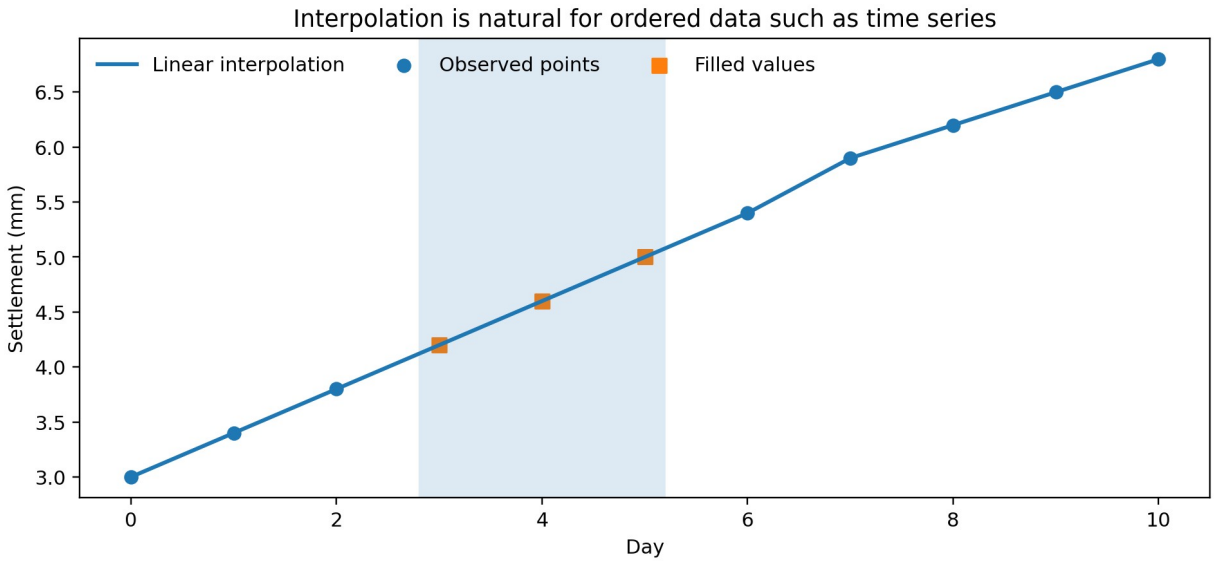

Figure 3. Interpolation is natural for ordered data such as time series. Source: author-created synthetic example, aligned with pandas interpolation usage [3].

5. Imputation with scikit-learn before machine learning

When the cleaned data will go into a machine-learning model, scikit-learn is a common choice. Its imputers are designed to learn imputation values from data and then apply the same transformation later.

SimpleImputer

SimpleImputer is the easiest sklearn option. It can fill numeric columns with a mean or median, and categorical columns with the most frequent value or a constant [4].

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

X = df[["cement", "water", "age", "slump"]]
y = df["strength"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

imp = SimpleImputer(strategy="median")
X_train_imp = imp.fit_transform(X_train)
X_test_imp = imp.transform(X_test)
```

Listing 6. Fit the imputer on the training data, then transform both training and test data [4], [6].

KNNImputer

KNNImputer fills a missing value by looking for rows that are similar in the other available features. It can work well when neighboring rows genuinely resemble each other, but it is slower than SimpleImputer.

```
from sklearn.impute import KNNImputer

imp = KNNImputer(n_neighbors=3)
X_train_imp = imp.fit_transform(X_train)
X_test_imp = imp.transform(X_test)
```

Listing 7. A minimal KNN imputation example. KNNImputer is part of sklearn.impute [4].

IterativeImputer

IterativeImputer estimates each incomplete feature using the other features in repeated rounds. It can be useful when several variables are strongly related, but it is a more advanced option. In the current stable scikit-learn documentation, IterativeImputer is still marked experimental [5].

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

imp = IterativeImputer(random_state=0)
X_train_imp = imp.fit_transform(X_train)
X_test_imp = imp.transform(X_test)
```

Listing 8. Keep this for later if you are just starting. The sklearn documentation still labels IterativeImputer as experimental [5].

Add a missing-value indicator when useful

Sometimes the fact that a value is missing is itself informative. You can keep that information with a simple indicator column.

```
df["slump_missing"] = df["slump"].isna().astype(int)
```

Listing 9. A tiny indicator variable. This can help when missingness itself carries information.

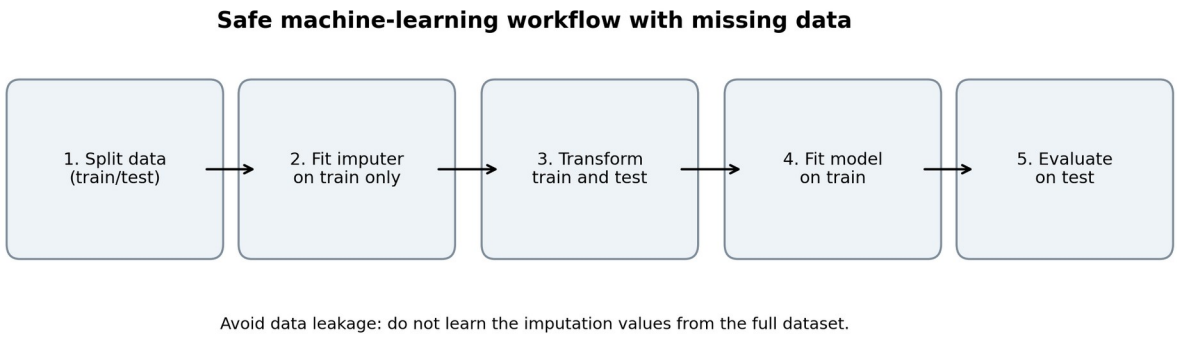

Figure 4. Safe workflow for machine learning with missing data. Split first, then fit the imputer on training data only. Source: author-created summary based on scikit-learn guidance [6].

6. Choosing a method quickly

Situation	Good first try	Reason
A few random missing numeric values	Median	Simple, robust, and fast
Categorical labels missing	Most frequent or 'Unknown'	Easy to explain and implement
Ordered sensor or time-series data	Interpolation	Respects order better than a single average
Rows with similar feature patterns	KNNImputer	Uses nearby rows instead of one global value
Several correlated numeric features	IterativeImputer	Uses relationships among variables
Very high missing fraction	Rethink the variable	Imputation may become too speculative

Table 4. A quick decision table for early coursework and project work.

Do not forget

- Report how many values were missing.
- Report which imputation method you used.
- Keep a copy of the raw data before filling values.
- Be extra cautious if the missingness is likely MNAR.
- For model evaluation, never let the test set influence the imputer.

7. A short notebook starter example

This small example is enough to start experimenting in Colab or Jupyter.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

# 1. load your data
```

```
df = pd.read_csv("civil_data.csv")

# 2. inspect missingness
print(df.isna().sum())

# 3. choose predictor columns and target
X = df[["cement", "water", "age", "slump"]]
y = df["strength"]

# 4. split first
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# 5. fit the imputer on training data only
imp = SimpleImputer(strategy="median")
X_train_imp = imp.fit_transform(X_train)
X_test_imp = imp.transform(X_test)

print("Ready for the next modeling step.")
```

Listing 10. A minimal end-to-end starter cell for numeric features.

References

[1] NCBI Bookshelf. Types of Missing Data. Managing Missing Data in Patient Registries.

[2] pandas documentation. Working with missing data.

[3] pandas documentation. DataFrame.interpolate.

[4] scikit-learn documentation. Imputation of missing values.

[5] scikit-learn documentation. IterativeImputer API reference.

[6] scikit-learn documentation. Common pitfalls and recommended practices.